

Project 2 Report

INFSCI 2591 - Algorithm Design

Abhay Kumara Sri Krishna Nandiraju
University of Pittsburgh

Spring 2026

1 Test Case Results

1.1 Dijkstra Algorithm using 2D Array

Test Case	Route	Distance (ft)	Path
1	192 → 163	819	[192, 157, 194, 158, 161, 163]
2	138 → 66	2728	[138, 162, 136, 159, 119, 116, 114, 112, 70, 110, 79, 108, 107, 103, 105, 85, 67, 66]
3	465 → 22	6738	[465, 377, 380, 372, 363, 364, 305, 247, 210, 233, 201, 170, 128, 98, 78, 41, 23, 22]

Table 1: Test Case results using Dijkstra's algorithm with 2D-array

1.2 Dijkstra Algorithm using Linked List

Test Case	Route	Distance (ft)	Path
1	192 → 163	819	[192, 157, 194, 158, 161, 163]
2	138 → 66	2728	[138, 162, 136, 159, 119, 116, 114, 112, 70, 110, 79, 108, 107, 103, 105, 85, 67, 66]
3	465 → 22	6738	[465, 377, 380, 372, 363, 364, 305, 247, 210, 233, 201, 170, 128, 98, 78, 41, 23, 22]

Table 2: Test Case results using Dijkstra's algorithm with Linked List

1.3 Floyd Algorithm using 2D Array

Test Case	Route	Distance (ft)	Path
1	192 → 163	819	[192, 157, 194, 158, 161, 163]
2	138 → 66	2728	[138, 162, 136, 159, 119, 116, 114, 112, 70, 110, 79, 108, 107, 103, 105, 85, 67, 66]
3	465 → 22	6738	[465, 377, 380, 372, 363, 364, 305, 247, 210, 233, 201, 170, 128, 98, 78, 41, 23, 22]

Table 3: Test Case results using Floyd's algorithm with 2D Array

1.4 Floyd Algorithm using Linked List

Test Case	Route	Distance (ft)	Path
1	192 → 163	819	[192, 157, 194, 158, 161, 163]
2	138 → 66	2728	[138, 162, 136, 159, 119, 116, 114, 112, 70, 110, 79, 108, 107, 103, 105, 85, 67, 66]
3	465 → 22	6738	[465, 377, 380, 372, 363, 364, 305, 247, 210, 233, 201, 170, 128, 98, 78, 41, 23, 22]

Table 4: Test Case results using Floyd's algorithm with Linked List

2 Time Performance Plots

2.1 Dijkstra's Algorithm

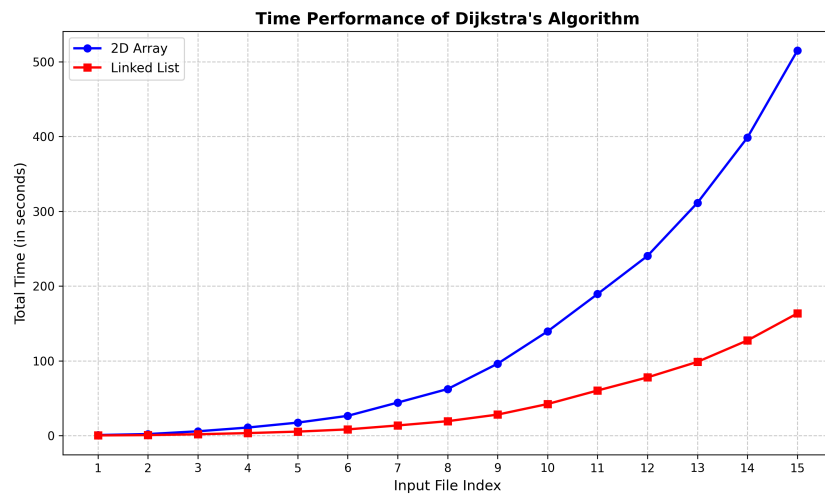


Figure 1: Time Taken by Dijkstra's Algorithm for 2D Array and Linked List Implementations

2.2 Floyd's Algorithm

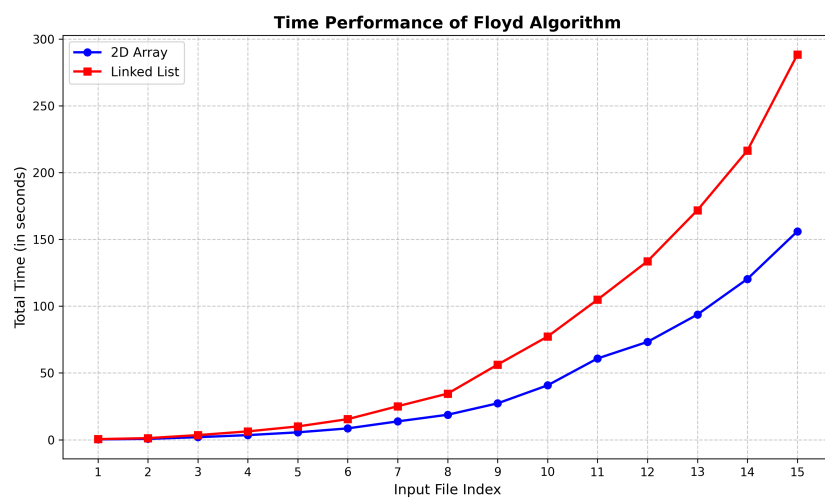


Figure 2: Time Taken by Floyd's Algorithm for 2D Array and Linked List Implementations

3 Time and Memory Usage

Memory usage was computed based on the data structures used in each algorithm assuming 64-bit architecture. The tables below show the time taken and memory used by each algorithm under both implementations.

File	Dijkstra (2D)	Dijkstra (LL)	Floyd (2D)	Floyd (LL)
1	0.7235	0.2276	0.2435	0.4475
2	2.0126	0.6304	0.6601	1.1999
3	5.7263	1.7914	1.9083	3.4535
4	10.7581	3.3236	3.4528	6.2476
5	17.3465	5.3035	5.5380	9.9565
6	26.4034	8.2791	8.4947	15.4086
7	44.1399	13.5275	13.7268	25.0734
8	62.2661	19.2583	18.7007	34.5197
9	96.1438	28.0619	27.2312	56.2458
10	139.3363	42.1884	40.7160	77.2402
11	189.3239	60.2658	60.8352	104.8465
12	240.3024	77.8398	73.2353	133.5842
13	311.3177	98.6026	93.6975	171.8189
14	398.5598	127.2308	120.3662	216.4860
15	514.8188	163.3865	155.9801	288.2544

Table 5: Time Performance (in seconds) across all 15 input files

File	Dijkstra (2D)	Dijkstra (LL)	Floyd (2D)	Floyd (LL)
1	315.82	17.35	942.19	642.16
2	621.55	24.34	1,857.23	1,257.84
3	1,213.16	35.09	3,629.11	2,447.98
4	1,807.97	43.73	5,411.25	3,643.27
5	2,450.54	51.43	7,336.88	4,933.40
6	3,281.12	59.72	9,826.31	6,599.85
7	4,477.68	70.49	13,413.09	9,000.00
8	5,592.35	79.13	16,754.77	11,234.95
9	7,200.93	90.44	21,577.50	14,459.52
10	9,454.19	103.19	28,333.59	18,974.01
11	11,515.21	114.27	34,513.64	23,103.23
12	13,655.06	125.53	40,930.36	27,390.51
13	16,336.68	137.75	48,971.95	32,761.73
14	19,209.52	150.15	57,587.25	38,515.63
15	23,060.45	164.92	69,136.08	46,227.13

Table 6: Memory Usage (in KB) across all 15 input files

The memory usage is calculated in the following way:

- **Memory usage in Dijkstra's algorithm with 2D array:**

Usage: adjacency matrix + distance array + previous array + visited array

Formula: $(N^2 \times 8) + 2 \times (N \times 8) + N$ bytes

- **Memory usage in Dijkstra's algorithm with linked list:**

Usage: Linked List size + distance array + previous array + visited array

Formula: $(N \times 8) + (E \times 24) + 2 \times (N \times 8) + N$ bytes

- **Memory usage in Floyd's algorithm with 2D array:**

Usage: 3x Matrices - Base, Dist, Next Node

Formula: $3 \times (N^2 \times 8) + 3 \times (N \times 8)$ bytes

- **Memory usage in Floyd's algorithm with linked list:**

Usage: Linked List size + 2x Matrices Dist, Next Node

Formula: $(N \times 8) + (E \times 24) + 2 \times (N^2 \times 8) + 2 \times (N \times 8)$ bytes

where $N = \text{nodes}$ and $E = \text{edges}$. Total memory usage in each case:

- **Dijkstra (2D):** 120,192.23KB(120.2MB)
- **Dijkstra (LL):** 1,267.53KB(1.27MB)
- **Floyd (2D):** 360,221.20KB(360.2MB)
- **Floyd (LL):** 241,191.21KB(241.2MB)

In terms of memory usage, Dijkstra's algorithm with linked list is significantly more efficient than the 2D array implementation, while Floyd's algorithm with linked list also uses less memory than the 2D array version, but both Floyd implementations use more memory than Dijkstra's.